

From manual to automated deployment

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

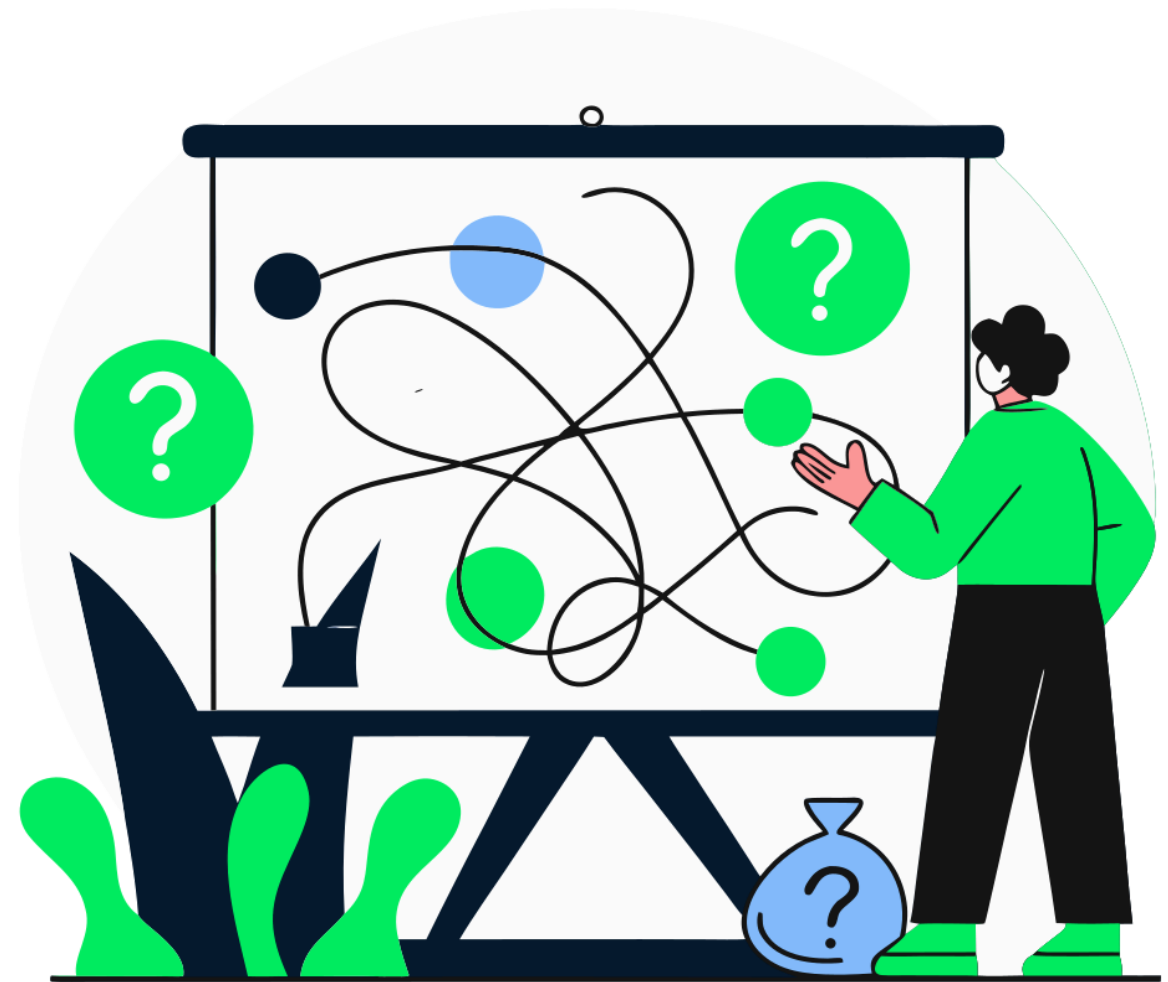
The manual deployment problem

- Configure jobs through the UI, one at a time
- Copy notebooks to production folders **manually**
- Settings **drift** between staging and production
- No way to **reproduce** a deployment from scratch

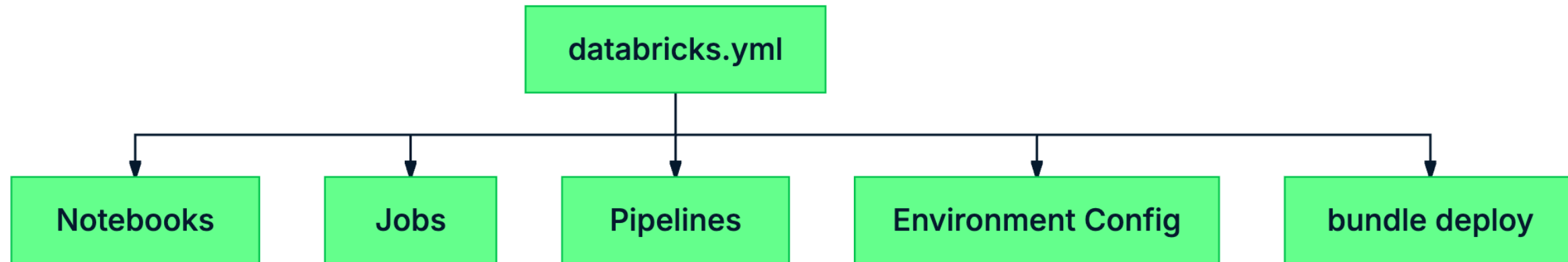


Why this matters at scale

- A typical project has **jobs, pipelines, clusters, permissions, and schedules**
- Each configured separately in the UI
- One team member leaves - their configuration knowledge **leaves with them**
- An audit asks "what changed last month?" - no clear answer



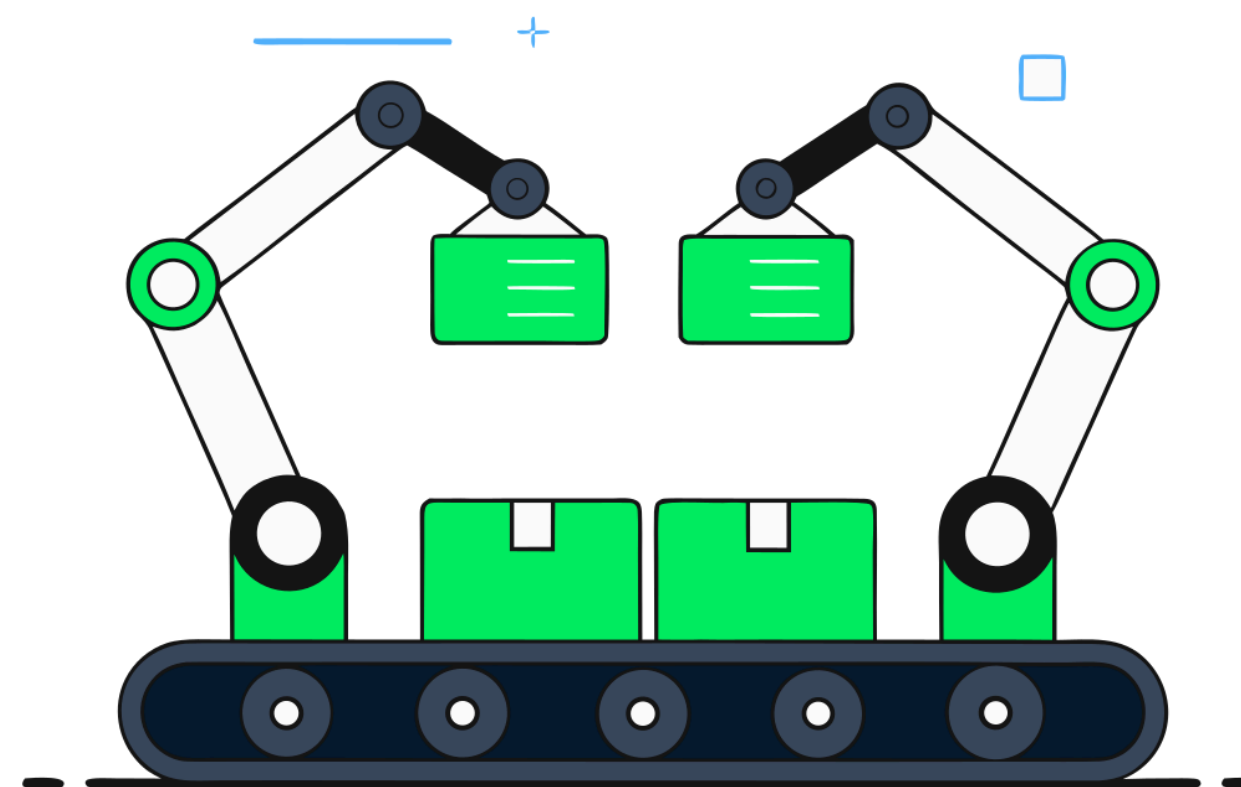
What are Databricks Asset Bundles?



- Everything in a **project folder**, versioned in **Git**, deployed with a **single CLI command**

DAB advantages

- **Reproducible** - deploy the same config every time
- **Version-controlled** - changes tracked in Git
- **Environment promotion** - dev → staging → prod
- **CI/CD integration** - automate deployments in pipelines



Traditional vs. DAB

	Traditional	DAB
Configuration	UI clicks	YAML files in Git
Reproducibility	Manual, error-prone	Automated, consistent
Environment promotion	Copy and reconfigure	<code>bundle deploy --target prod</code>
Rollback	Recreate from memory	<code>git revert</code> + redeploy
Collaboration	One person at a time	Full Git workflow

Summary

- Manual UI deployment is **fragile** and doesn't scale
- **Databricks Asset Bundles** describe your project in YAML
- Benefits: **reproducibility, version control, CI/CD, environment promotion**
- Everything lives in Git - one source of truth for your deployment

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Inside an asset bundle

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

Bundle project structure

```
my_project/  
|-- databricks.yml  
|-- src/  
|   |-- etl_pipeline.py  
|   |-- data_quality.py  
|-- resources/  
|   |-- jobs/  
|   |   |-- nightly_etl.yml  
|   |-- pipelines/  
|       |-- sales_pipeline.yml  
|-- tests/  
    |-- test_etl.py
```

- `databricks.yml` - the **central config file**
- `src/` - your **notebooks and code**
- `resources/` - **job and pipeline definitions**
- `tests/` - **optional test files**

The databricks.yml file

```
bundle:  
  name: sales_analytics  
  
workspace:  
  host: https://myworkspace.databricks.com  
  
targets:  
  dev:  
    default: true  
    workspace:  
      root_path: /Users/me/dev  
  production:  
    workspace:  
      root_path: /Shared/production  
    permissions:  
      - level: CAN_MANAGE  
        group_name: data_engineers
```

Defining resources

databricks.yml (top)

Resources - Jobs - Pipelines
- Dashboards

Code - Notebooks - Python
scripts

Targets - dev - staging -
production

```
resources:  
  jobs:  
    nightly_etl:  
      name: "Nightly ETL Pipeline"  
      schedule:  
        quartz_cron: "0 0 2 * * ?"  
      tasks:  
        - task_key: ingest  
          notebook_task:  
            notebook_path: src/etl.py
```

Bundle CLI commands

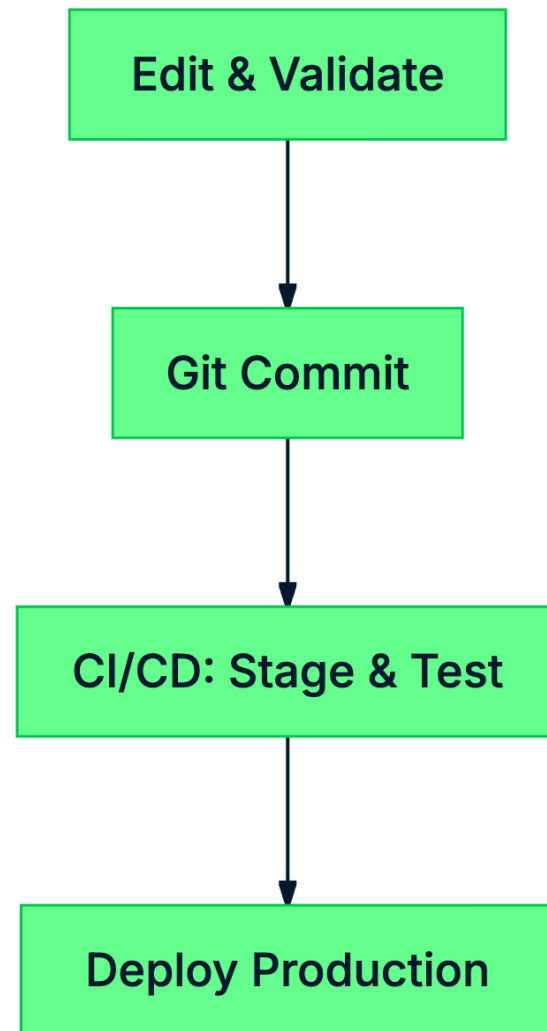
Command	Purpose
<code>bundle validate</code>	Check config for errors
<code>bundle deploy</code>	Deploy to a target
<code>bundle run</code>	Trigger a deployed job
<code>bundle destroy</code>	Remove deployed resources

```
# Validate before deploying
databricks bundle validate
```

```
# Deploy to production
databricks bundle deploy \
  --target production
```

```
# Trigger a run
databricks bundle run nightly_etl
```

Deployment workflow



- **Local development** - edit code and validate configs
- **Version control** - commit changes to Git
- **CI/CD automation** - deploy to staging and run tests
- **Production promotion** - deploy with confidence

Summary

- `databricks.yml` defines your project, targets, and resources
- **Resources** include jobs, pipelines, and dashboards
- **Targets** map to environments (dev, staging, production)
- CLI commands: `validate` , `deploy` , `run` , `destroy`

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Putting it all together

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

The scenario

- You're a data engineer starting a **new project** at a retail company
- Goal: set up compute, verify the data pipeline, check governance, and prepare for deployment



Step 1: Choose the right compute

- The pipeline runs **nightly** on a schedule
- No interactive development needed
- A **jobs cluster** with LTS runtime is the right call
- Set autoscaling (2–6 workers) for variable data volume

	All-Purpose	Jobs Cluster
Mode	Interactive	Automated
Management	Manual	Auto-terminates
Cost	Higher (idle time)	Cost-optimized

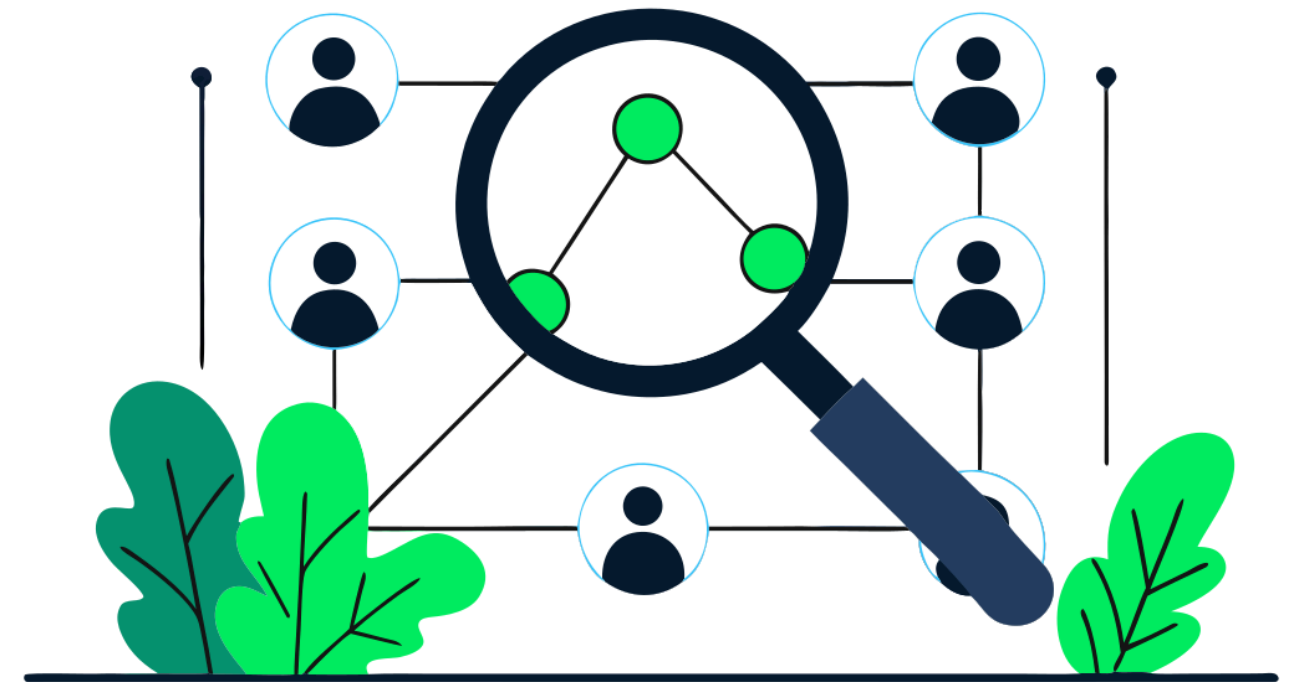
Step 2: Verify the medallion pipeline

```
SELECT COUNT(*) FROM bronze_sales;  
-- 1,247,832 rows  
  
SELECT COUNT(*) FROM silver_sales;  
-- 1,189,456 rows (nulls removed)  
  
SELECT COUNT(*) FROM gold_daily_revenue;  
-- 365 rows (daily aggregates)
```

- Bronze: **raw** data, highest row count
- Silver: **cleaned**, nulls and duplicates removed
- Gold: **aggregated** for business reporting
- Row counts decrease as quality increases

Step 3: Check governance

- Navigate to Unity Catalog and inspect the **gold_daily_revenue** table
- Trace **lineage** upstream to confirm it reads from **silver_sales**
- Verify **access controls** - only the analytics team has SELECT on gold
- Confirm **Delta Sharing** is configured for the external partner



Step 4: Prepare for deployment

- Review the `databricks.yml` Asset Bundle
- Confirm the nightly job resource is defined
- Check that **production target** points to the right workspace path
- Run `bundle validate` before deploying

```
targets:
  production:
    workspace:
      root_path: /Shared/production
resources:
  jobs:
    nightly_sales_etl:
      schedule:
        quartz_cron: "0 0 3 * * ?"
```

Summary

- **Compute:** match cluster type to workload (jobs cluster for automation)
- **Data:** verify medallion layers are flowing correctly
- **Governance:** trace lineage, check access, confirm sharing
- **Deployment:** review the Asset Bundle, validate, deploy

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Congratulations!

INTRODUCTION TO DATABRICKS LAKEHOUSE



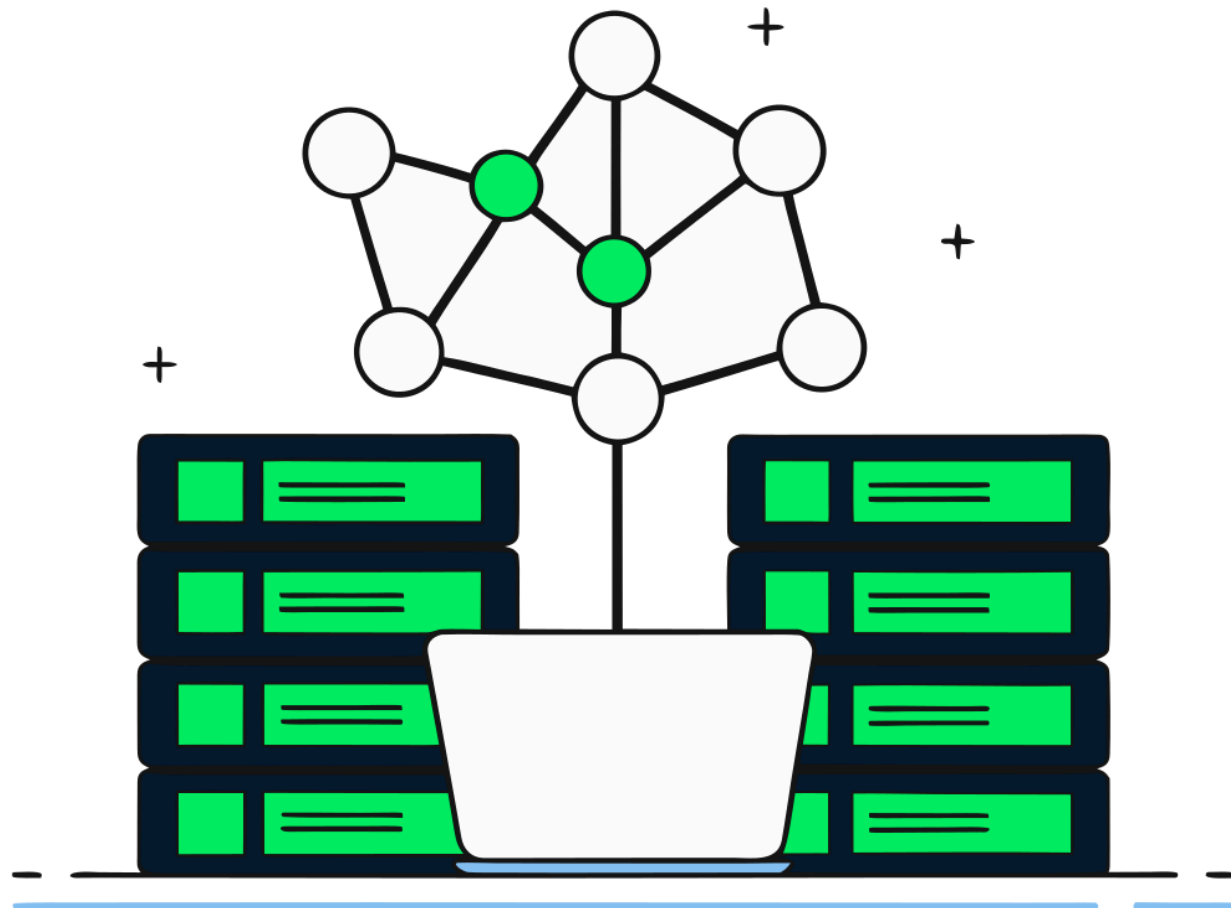
Gang Wang
Senior Data Scientist

Chapter 1: The Lakehouse Paradigm

- The **lakehouse** combines lake flexibility with warehouse reliability
- The **medallion architecture** organizes data: bronze → silver → gold
- The **control plane** is managed by Databricks; the **data plane** lives in your cloud



Chapter 2: Compute and Notebooks



- **All-purpose clusters** for interactive work; **jobs clusters** for automation
- **Autoscaling, auto-termination, and policies** keep costs in check
- **Notebooks** support multiple languages with magic commands
- **Databricks Repos** brings full Git integration to the workspace

Chapter 3: Governance and Sharing

- **Unity Catalog** provides centralized governance and automatic **lineage** tracking
- **Delta Sharing** gives external partners live access to data - no copies
- **Native** sharing for Databricks-to-Databricks; **open protocol** for any platform
- **Lakehouse Federation** queries external databases without moving data



Chapter 4: Deployment



- **Databricks Asset Bundles** replace manual UI deployment with code
- `databricks.yml` defines your project, resources, and targets
- CLI commands: `validate` , `deploy` , `run` , `destroy`
- Everything lives in **Git** - reproducible, auditable, automated

Where to go next

- **Introduction to Databricks SQL** - SQL analytics and warehousing
- **Data Engineering with Databricks** - production ETL pipelines
- **Databricks Concepts** - end-to-end workflows across personas



Thank you!

INTRODUCTION TO DATABRICKS LAKEHOUSE

